

Code Explanation

Delta Live Tables Code Explanation

This code is used to create a Delta Live Table (DLT) pipeline that ingests data from CSV files, performs data cleaning and transformation, and creates aggregated tables. The pipeline consists of multiple tables, each with its own specific functionality.

Overview

The code defines a DLT pipeline that reads data from CSV files, cleans and transforms the data, and creates multiple tables with different levels of aggregation. The pipeline consists of bronze, silver, and aggregated tables.

Step 1: Importing Necessary Libraries and Defining Source Data Paths

This step imports the necessary libraries, including PySpark SQL functions and Delta Tables. It also defines the source data paths for customer and order data.

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from delta.tables import DeltaTable
import dlt

# Define source data paths
CUSTOMER_PATH = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata"
ORDER_PATH = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata"
```

Step 2: Defining Catalog and Schema

This step defines the catalog and schema for the DLT pipeline.

```
# Define catalog and schema
CATALOG = "gen_ai_poc_databrickscoe"
SCHEMA = "sdlc_wizard"
```

Step 3: Creating Bronze Tables

This step creates two bronze tables, `customer\_bronze` and `order\_bronze`, which read data from the source CSV files.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data from source"
)
def customer_bronze():
    return spark.read.format("csv").option("header", "true").option(
        "inferSchema", "true").load(CUSTOMER_PATH)

@dlt.table(
    name="order_bronze",
    comment="Raw order data from source"
)
```

```
def order_bronze():
    return spark.read.format("csv").option("header", "true").option(
        "inferSchema", "true").load(ORDER_PATH)
```

Step 4: Creating Silver Tables

This step creates two silver tables, `customer\_silver` and `order\_silver`, which clean and transform the data from the bronze tables.

```
@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data with duplicates and nulls removed"
)
def customer_silver():
    # Read from bronze layer
    df = dlt.read("customer_bronze")

    # Remove nulls and duplicates
    return df.dropDuplicates(["CustId"])
               .filter(F.col("CustId").isNotNull() &
                       F.col("Name").isNotNull() &
                       F.col("EmailId").isNotNull() &
                       F.col("Region").isNotNull())

@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    # Read from bronze layer
    df = dlt.read("order_bronze")

    # Remove nulls and duplicates, and add TotalAmount column
    return df.dropDuplicates(["OrderId"])
               .filter(F.col("OrderId").isNotNull() &
                       F.col("ItemName").isNotNull() &
                       F.col("PricePerUnit").isNotNull() &
                       F.col("Qty").isNotNull() &
                       F.col("Date").isNotNull() &
                       F.col("CustId").isNotNull())
               .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
```

Step 5: Creating ordersummary Table with SCD Type 2 Implementation

This step creates the `ordersummary` table, which combines customer and order data and implements Slowly Changing Dimension (SCD) Type 2.

```
@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table combining customer and order data",
    table_properties={
        "quality": "silver",
        "delta.enableChangeDataFeed": "true"
    }
)
```

```

    }
)
def ordersummary():
    # Read the latest customer and order data
    customer_df = dlt.read("customer_silver")
    order_df = dlt.read("order_silver")

    # Join customer and order data
    joined_df = order_df.join(customer_df, "CustId", "inner")

    # Select columns as per the required schema
    current_data = joined_df.select(
        "CustId", "Name", "EmailId", "Region",
        "OrderId", "ItemName", "PricePerUnit", "Qty", "Date"
    )

    # ... (rest of the code for SCD Type 2 implementation)

```

Step 6: Creating customeraggregatespend Table

This step creates the `customeraggregatespend` table, which aggregates customer spending by name and date.

```

@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date"
)
def customeraggregatespend():
    # Read from ordersummary
    ordersummary_df = dlt.read("ordersummary")

    # Join with order_silver to get TotalAmount
    order_df = dlt.read("order_silver")

    joined_df = ordersummary_df.join(
        order_df,
        ["OrderId"],
        "inner"
    )

    # Aggregate TotalAmount by Name and Date
    aggregated_df = joined_df.groupBy("Name", "Date")
        .agg(F.sum("TotalAmount").alias("TotalAmount"))

    return aggregated_df

```